(i) 5 (ii) the address of test

(iii) 2 (iv) ree

v

2147483648x2= 4294967296

Size of struct is 16 bytes.
Part1=~~2147483647~~, Part2=~~2147483648~~

4294967295 4294967293

5+3+4+4

-1 ⇒ 111……1

32個1

```
int main(void)
{
    int a,b;

    printf ("Enter two integers: ");
    scanf ("%d%d", &a, &b);
    if (a-b)
    printf ("Not Equal");
    else
    printf ("Equal");
    return 0;
}
```

```
int main(void)
{
    int a, b;
    printf("Enter two integers: ");
    scanf("%d%d", &a, &b);

    if (a ^ b)
        printf("Not Equal");
    else
        printf("Equal");
    return 0;
}
```

```
unsigned encode (unsigned this1)
{
    unsigned this2;
    this2=this1;

}
```

```
unsigned encode(unsigned this1)
{
    unsigned h0 = (this1 & 0x0000000F) << 8;    // h0 -> bits 8-11
    unsigned h1 = (this1 & 0x000000F0) << 4;    // h1 -> bits 12-15
    unsigned h2 = (this1 & 0x00000F00);         // h2 -> bits 8-11 (保持位置)
    unsigned h3 = (this1 & 0x0000F000) >> 4;    // h3 -> bits 16-19
    unsigned h4 = (this1 & 0x000F0000) << 12;   // h4 -> bits 28-31
    unsigned h5 = (this1 & 0x00F00000) << 4;    // h5 -> bits 20-23 → bits 24-27
    unsigned h6 = (this1 & 0x0F000000) >> 24;   // h6 -> bits 0–3
    unsigned h7 = (this1 & 0xF0000000) >> 28;   // h7 -> bits 4–7

    unsigned result = h0 | h1 | h2 | h3 | h4 | h5 | (h6 << 0) | (h7 << 4);
    return result;
}
```

first->next=this1

this1.next=NULL

```
//
//  main.c
//  Mock test-6
//
//  Created by Hanzhong Zhang on 22/5/25.
//

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    typedef struct employee {
        char name[14];
        int emp_no;
        char gender;
        float rate, hr;
        struct employee *next;
    } node;

    float pay, total = 0;
    FILE *indata;
    node *temp, *first = NULL, *last = NULL, *new_node;

    char name_buf[14];
    int emp_no;
    char gender;
    float rate, hr;

    indata = fopen("wage.inf", "r");
    if (indata == NULL)
    {
        printf("\n wage.inf cannot be found.");
        exit(1);
    }

    printf("\n Employee No. Hours Worked Hourly Rate Wage");
    printf("\n (hr) ($/hr) ($)");
    printf("\n ================================================");

    while ((fgets(name_buf, 14, indata) != NULL) &&
           (fscanf(indata, "%d %c%f%f%*c", &emp_no, &gender, &rate, &hr) == 4))
    {
        new_node = (node *)malloc(sizeof(node));
```

```
    if (new_node == NULL)
    {
        printf("Memory allocation failed.\n");
        exit(1);
    }

    strncpy(new_node->name, name_buf, 13);
    new_node->name[13] = '\0';
    new_node->emp_no = emp_no;
    new_node->gender = gender;
    new_node->rate = rate;
    new_node->hr = hr;
    new_node->next = NULL;

    if (first == NULL)
        first = new_node;
    else
        last->next = new_node;
    last = new_node;
}

temp = first;
while (temp != NULL)
{
    printf("\n %4d %6.2f %6.2f ",
          temp->emp_no, temp->hr, temp->rate);
    if (temp->hr <= 40)
        pay = temp->hr * temp->rate;
    else
        pay = temp->rate * 40 + temp->rate * (temp->hr - 40) * 1.5;
    printf(" %6.2f ", pay);
    total += pay;
    temp = temp->next;
}
printf("\n\n                          Total wages = $%-8.2f", total);

// Free the allocated memory
temp = first;
while (temp != NULL)
{
    node *to_free = temp;
    temp = temp->next;
    free(to_free);
}
```

```
    fclose(indata);
    return 0;
}
```

(i) S (ii) address (iii) v (iv) ree

↓

of "one

usigned int: $0 \sim 2^{32}-1 = 0 \sim 4294967295$

共2^{32}个数

int: $-2^{31} \sim 2^{31}-1$ (0算正数)

正负数目相等.

5+3+8=16

16

0-1 会自动模2^{32} $0-1 \equiv 2^{32}-1 \pmod{2^{32}}$

3. Re-write the following C programming **without** using the ==, >, >=, <, <= and ! operators to compare two numbers.

```
void main ()
{
  int a, b;

  printf ("Enter two integers: ");
  scanf ("%d%d", &a, &b);

  if (a==b)
    printf ("Equal");
  else
    printf ("Not Equal");
}
```

(5 marks)

Answer:

```
void main ()
{
  int a, b;

  printf ("Enter two integers: ");
  scanf ("%d%d", &a, &b);

  if (a^b)     // 0,1 or 1,0
    printf ("Not Equal");
  else         // 0,0 or 1,1
    printf ("Equal");
}
```

66 B

x1=(n&0x0000ffff)<<8;
x2=(n&0x00ff0000)<<12;
x3=(n&0xff000000)<<4;
x4=(n&0x00ff0000)>>20;
x5=(n&0xff000000)>>28;

x2|x3|x1|x4|x5

5. Bit-wise transformation technique can be used to scramble data for security purpose. Assume the following declarations:

```
unsigned this1, result;
```

Let the bits of the unsigned integer `this1` be labeled as $b_{31}b_{30}$ ...$b_2b_1b_0$, and let its contents be $(h_7h_6h_5h_4h_3h_2h_1h_0)_{(16)}$, i.e., $h_7h_6h_5h_4h_3h_2h_1h_0$ is a hexadecimal number, such as $345abd7f_{(16)}$, $ffffffff_{(16)}$, etc.

Our encryption goes as follows. Given a hexadecimal number $h_7h_6h_5h_4h_3h_2h_1h_0$, the encryption will produce the `result` with $h_4h_5h_3h_2h_1h_0h_6h_7$.

(i) Complete the encode function that accepts an unsigned integer as an input argument, and performs the above transformation on the unsigned integer by bit manipulation operators. The result is then returned to the caller. The function header is given as follows:

```
unsigned encode (unsigned this1)
```

For example, if this1 = $fedcba98_{(16)}$, the encode function should return $cdba98ef_{(16)}$. **(30 marks)**

Answer:

```
# include<stdio.h>

unsigned encode (unsigned this1)
{
  unsigned x03, x4, x5, x6, x7;

  x03 = (this1&0x0000ffff) << 8;   x4 =  (this1&0x000f0000) << 12;
  x5 =  (this1&0x00f00000) << 4;  x6 =  (this1&0x0f000000) >> 20;
  x7 =  (this1&0xf0000000) >> 28;

  return (x4 | x5 | x03 | x6 |x7);
}
```

$h_7h_6h_5h_4h_3h_2h_1h_0$

$h_4h_5h_3h_2h_1h_0h_6h_7$

(ii) Complete the decode function to perform the inverse transformation to recover the original unsigned integer by bit manipulation operators. The input parameter is the encoded unsigned integer. The original unsigned integer should be returned to the caller. The function header is given as follows:

```
unsigned decode (unsigned result)
```

For example, if result = $45321067_{(16)}$, the decode function should return $76543210_{(16)}$. **(15 marks)**

Answer:

```
unsigned decode (unsigned result)
{
  unsigned x03, x4, x5, x6, x7;

  x03 = (result & 0x00ffff00) >> 8;
  x4 =  (result & 0xf0000000) >> 12;
  x5 =  (result & 0x0f000000) >> 4;
  x6 =  (result & 0x000000f0) << 20;
  x7 =  (result & 0x0000000f) << 28;

  return (x4 | x5 | x03 | x6 |x7);
}
```

$h_7h_6h_5h_4h_3h_2h_1h_0$

$h_4h_5h_3h_2h_1h_0h_6h_7$

```
main ()
{
  unsigned this1, result;

  printf ("\n");
  printf (" Enter an unsigned hexadecimal number with 8 digits: ");
  scanf ("%x", &this1);
  result = encode (this1);
  printf (" Encoded form of user input: %x", result);
  printf ("\n Decoded form of encoded number: %x", decode (result));

  return 0;
}
```